```
print http_build_query($data);
```

```
Output:
var1=%26val1&var2=%26val2
```

Notice how the values &val1 and &val2 have been encoded to %26val1 and %26val2? If this wasn't done, we'd break the syntax of the query string. Let's see what happens if those %26 sequences were just literal & characters:

```
parse_str('var1=&val1&var2=&val2', $mess);
var_dump($mess);
```

```
Output:
array(4) {
  ["var1"]=>
  string(0) ""
  ["val1"]=>
  string(0) ""
  ["var2"]=>
  string(0) ""
  ["val2"]=>
  string(0) ""
}
```

Ugh. What a mess. We end up with four parameters and no values. The parser reads var1=&, which is interpreted as "the var1 parameter has no value," next it reads val1&, and despite the lack of an = symbol, this is also treated as a parameter with no value. And so on. A similar problem exists if we encoded all the & characters:

```
parse_str('var1=%26val1%26var2=%26val2', $mess);
var_dump($mess);
```

```
Output:
array(1) {
  ["var1"]=>
  string(16) "&val1&var2=&val2"
}
```